NLP MASTERCLASS

Concept-First Engineering Notes and Technical Interview Guide

Six Core Modules | 60 Interview Q&As | Practical Python

October 2026

How to use this guide

Study pattern: for each technique, explain the input-to-output flow aloud, redraw the architecture from memory, run the code, then answer the five scenario questions without looking. Prefer engineering reasoning and failure analysis over memorized definitions.

Module	Representation / core idea	Good starting point when…	Main limitation
1. Text preprocessing	Raw text to reliable tokens	Data is inconsistent, multilingual or noisy	Destructive normalization can erase meaning
2. BoW / TF-IDF	Sparse lexical counts and weights	Labels are limited; speed and explainability matter	Poor semantic generalization
3. Word2Vec / FastText	Dense static word vectors	Need compact, trainable semantic features	Meaning does not adapt to surrounding sentence
4. RNN / LSTM	Sequential hidden states	Ordered signals, small sequence models	Serial computation, long-range limitations
5. Transformers	Context-dependent attention	Contextual classification and generation	Compute, memory, context and hallucination risks
6. LLM fine-tuning	Adapt pretrained behavior / weights	Repeated domain-specific behavior is required	Data quality, cost and regression risk

Notation without derivations: a *token* may be a word, subword or punctuation; *embedding* is a learned numeric vector; *logits* are unnormalized model scores; *mask* determines which tokens may participate in computation; *context window* is the model's maximum supported token span. Production claims below describe **design patterns**; the deliberately tiny datasets in code are educational and **not** evidence of real-world model quality.

1. Text Preprocessing and Tokenization

1.1 Core Algorithm and Workflow Intuition

How it works — the full journey. Start with raw sources such as support tickets, PDFs, email, chat, logs or transcripts. Decode to a consistent character encoding, normalize safe Unicode variants, handle boilerplate and sensitive fields, detect language if needed, split into sentences or spans, tokenize, and create a representation appropriate to the downstream task. For a classical model, optional lowercase/lemmatization/stopword removal can help. For BERT/GPT, usually feed minimally cleaned text to the **model's own pretrained tokenizer**, preserving punctuation, case and subword boundaries expected by the checkpoint.

Key operations and engineering decisions:

- **Unicode normalization:** NFC can combine canonically equivalent character sequences. NFKC performs compatibility transformations and may change distinctions; use only after domain checks. Never silently strip Indic characters or accents.
- **Tokenization:** word tokenizers split by language-specific conventions; subword tokenizers (WordPiece, BPE, Unigram) split rare words into known fragments. Character tokenization avoids unknown characters but lengthens sequences.
- **Normalization:** lowercase only when case does not encode useful meaning; US versus us, SKU identifiers and person names can matter.
- **Stopwords / lemmatization / stemming:** potentially reduce vocabulary for sparse models; never assume these help transformers. Preserve **not/no/never**, numbers, units, negation, dates, product codes and legal clauses when task-relevant.
- **Privacy and quality:** redact PII with an auditable process, deduplicate near-identical records and preserve raw-to-cleaned traceability. Split data by user/document/time **before** fitting any statistics or vocabulary to avoid leakage.

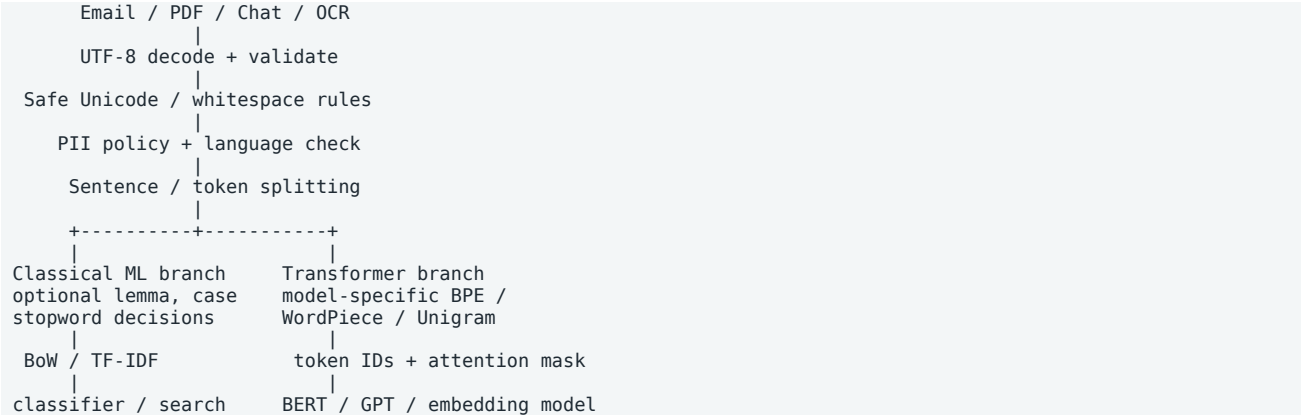
Pros. Makes noisy corpora consistent, reduces sparse vocabulary, improves retrieval recall when normalization matches the query, and adds inspectable safeguards.

Cons / limits. Aggressive cleaning can reverse sentiment ("not good" -> "good"), destroy ID fields or word order, conflate languages, corrupt offsets used for NER/highlighting, and create train-serving skew. Spelling correction can wrongly rewrite domain vocabulary.

When to choose — industry use cases:

1. **Customer-support routing:** normalize channel-specific boilerplate and punctuation, but preserve negation and product IDs.
2. **Enterprise RAG ingestion:** remove repeated headers/footers and track page/document offsets without flattening tables or citations.
3. **Compliance / PII screening:** consistent tokenization and reversible location metadata are essential for entity extraction and audit.

1.2 Architectural Blueprint



Whiteboard tip: annotate a danger sign next to not, names, emojis, URLs, SKUs and multilingual characters. Explain which are retained and why.

1.3 Production-Ready Simple Code

Dependencies: pip install spacy scikit-learn (no external spaCy language model required for this tokenizer-only example).

```
import spacy
from sklearn.model_selection import train_test_split
from sklearn.pipeline import Pipeline
from sklearn.feature_extraction.text import TfidfVectorizer
from sklearn.linear_model import LogisticRegression
from sklearn.metrics import classification_report

nlp = spacy.blank("en")  # Lightweight, rules-based tokenizer

# Use the same transformation at training and inference.
def clean(text: str) -> str:
    tokens = []
    for t in nlp(text):
        if not t.is_alpha:
            continue  # For this *sentiment* demo only; preserve IDs in RAG
        word = t.text.lower()
        # Keep negation; removing it can reverse the label.
        if not t.is_stop or word in {"not", "no", "never"}:
            tokens.append(word)
    return " ".join(tokens)

texts = [
    "Great service and quick delivery", "Very helpful support team",
    "Loved the simple setup", "Excellent and reliable product",
    "Fast response and friendly staff", "The app works perfectly",
    "Not good: delivery was late", "Terrible service and long wait",
    "The product is broken", "Never received my order",
    "Bad customer support", "Not happy with this purchase",
]
labels = [1] * 6 + [0] * 6
X_train, X_test, y_train, y_test = train_test_split(
    texts, labels, test_size=0.33, random_state=42, stratify=labels
)
model = Pipeline([
    ("tfidf", TfidfVectorizer(preprocessor=clean, ngram_range=(1, 2))),
    ("clf", LogisticRegression(max_iter=500)),
])
model.fit(X_train, y_train)  # Learns vocabulary on train only
print(classification_report(y_test, model.predict(X_test), zero_division=0))
print(clean("I am NOT satisfied with the delivery!"))
```

Production hardening: benchmark with and without cleaning; persist the entire pipeline; add tests for contractions, emojis, accents, Hindi/English mixing, PII and domain identifiers; evaluate on an untouched time-split holdout.

1.4 Top 5 NLP Interview Questions

Q1. A classifier performs well offline, but quality drops in production after preprocessing changes. How do you debug it?
Answer: Compare raw and transformed examples from train versus serving, including tokenizer version, Unicode rules, stopword list and model artifacts. Look for changes in token counts, OOV rate, truncation and class-specific failures; replay a fixed regression set through both paths. Deploy preprocessing and model as one versioned artifact, then canary-test before rollout.

Q2. Why can stopword removal or stemming reduce accuracy?
Answer: Stopwords are not always uninformative: removing “not” reverses sentiment, while “may” or “shall” can define legal intent. Stemming can merge distinct technical terms, damage named entities and degrade citation matching. Ablate each operation and keep only changes that improve a held-out task metric.

Q3. Your tokenizer breaks product ID AB-120/4 into pieces and retrieval fails. What changes?
Answer: Treat product identifiers as protected spans before generic tokenization; add domain-specific tokenizer exceptions or use a separate normalized exact-match field. Combine exact keyword matching with semantic retrieval, and log both the original and normalized forms. Test query and document sides for identical canonicalization.

Q4. How do you handle OOV words in classical versus transformer NLP?
Answer: Fixed word vocabularies may map unseen words to UNK or drop them, causing sparse or unusable features. Character n-grams and FastText subwords help; pretrained BPE/WordPiece tokenizers usually break rare words into known subwords. However, known subword pieces do **not** guarantee the model understands a new domain term, so evaluate domain-shift examples.

Q5. What is the difference between tokenization, stemming and lemmatization?
Answer: Tokenization identifies units and their boundaries. Stemming heuristically chops affixes and may produce nonwords, whereas lemmatization uses vocabulary/morphology (often POS-sensitive) to return base forms. Choose them for specific downstream objectives, not as universally required preprocessing steps.

---

2. Bag-of-Words, TF-IDF and Lexical Baselines

2.1 Core Algorithm and Workflow Intuition

How it works. Bag-of-Words creates a vocabulary from the training corpus and represents each document as counts (or 0/1 indicators) of vocabulary terms. TF-IDF modifies those weights: terms frequent in a document matter more, while terms seen in almost every document are down-weighted. Both produce **sparse** document-term matrices. Fit a linear classifier, logistic regression, linear SVM or a similarity search system over that matrix.

Mechanics worth explaining at interview:

1. Build vocabulary on the **training set only**; choose word or character n-grams, minimum document frequency and maximum vocabulary size.
2. Vectorize a document; “refund refund today” gives higher term frequency for `refund`.
3. Apply inverse-document-frequency weighting and usually vector normalization; cosine similarity then compares lexical direction rather than raw document length.
4. For a classifier, learned coefficients show which terms correlate with labels. For search, TF-IDF retrieves lexical overlap; **BM25** is a distinct probabilistic term-ranking method with frequency saturation and length normalization.

Pros. Extremely fast on CPU; surprisingly competitive on small/medium labeled text classification; interpretable and controllable; no GPU or massive pretrained checkpoint; exact terminology strongly influences the score.

Cons / limits. Unigrams ignore word order (“dog bites man” vs “man bites dog”); n-grams partially fix this but enlarge dimensionality. Synonyms do not automatically match (“reimburse” vs “refund”); unseen tokens add no learned feature; vocabulary drift and language variation affect recall.

When to choose:

1. **Ticket categorization / spam filtering** with thousands rather than millions of labeled examples.
2. **Exact terminology search** in legal, pharmaceutical or SKU-heavy corpora, especially as a BM25/hybrid-retrieval baseline.
3. **Low-latency, interpretable classifiers** for constrained CPU deployments.

2.2 Architectural Blueprint

```
Documents                    Learned TRAIN vocabulary
D1: "refund order" ------> [order, refund, delay, approved]
D2: "order delay"                      |
D3: "refund approved"                  v
                         Sparse term-by-document matrix
                         D1: [1, 1, 0, 0]
                         D2: [1, 0, 1, 0]
                         D3: [0, 1, 0, 1]
                                       |
                              TF-IDF weighting
                                       |
                           Linear model / cosine rank
```

Sketch: Draw an additional word-bigram column `not_good` and show why that partially preserves local order.

2.3 Production-Ready Simple Code

Dependencies: `pip install scikit-learn`

```
from sklearn.feature_extraction.text import CountVectorizer, TfidfVectorizer
from sklearn.linear_model import LogisticRegression
from sklearn.model_selection import train_test_split
from sklearn.pipeline import Pipeline
from sklearn.metrics import accuracy_score, f1_score

texts = [
    "refund my purchase", "please cancel order", "payment failed",
    "charged twice for order", "return damaged item", "need invoice refund",
```

```
    "cannot access account", "login password reset", "app crashes",
    "dashboard not loading", "authentication error", "reset account password",
]
y = [1] * 6 + [0] * 6  # 1=billing, 0=technical

X_tr, X_te, y_tr, y_te = train_test_split(
    texts, y, test_size=0.33, stratify=y, random_state=7
)

bow = CountVectorizer(ngram_range=(1, 2))
X_bow = bow.fit_transform(X_tr)  # Show the sparse count baseline
print("BoW shape:", X_bow.shape)

pipe = Pipeline([
    ("tfidf", TfidfVectorizer(ngram_range=(1, 2), sublinear_tf=True)),
    ("classifier", LogisticRegression(max_iter=500, class_weight="balanced")),
])
pipe.fit(X_tr, y_tr)
pred = pipe.predict(X_te)
print("Accuracy:", accuracy_score(y_te, pred))
print("Macro F1:", f1_score(y_te, pred, average="macro"))
print("Sample prediction:", pipe.predict(["please refund my order"]))
```

Production hardening: use time- or customer-group holdouts, tune n-grams/min_df via cross-validation, calibrate scores if used as probabilities, and compare with a transformer before accepting higher model latency.

2.4 Top 5 NLP Interview Questions

Q1. Why is TF-IDF often stronger than raw counts, but not always?
Answer: It reduces the influence of corpus-common terms, making rarer discriminative words more prominent. Raw counts can be better when frequent terms are directly predictive (e.g., repeated abuse signals) or when IDF estimates are unstable on a tiny corpus. Compare both under the same train/validation split; neither encodes semantic understanding.

Q2. How would you fix “not approved” being confused with “approved”?
Answer: Add word bigrams or a feature for negation scope; do not remove “not” as a stopword. Character n-grams also help with misspellings, but neither solves arbitrary long-range syntax. If these remain major failure modes, use a context-aware transformer.

Q3. How do you handle unseen words at inference?
Answer: A learned word vocabulary ignores OOV terms, so monitor unknown-token coverage. Character n-grams, hashing vectorizers and periodic vocabulary refits can help; refitting changes feature indices, so retrain the classifier and ship both objects together. For semantic synonyms, consider dense embeddings or hybrid models.

Q4. A TF-IDF search engine returns long irrelevant documents. What do you change?
Answer: Check vector normalization, stopword behavior, boilerplate duplication and query expansion; use document length-aware BM25 as a baseline. Segment long documents into coherent chunks, preserve metadata and rerank. Evaluate Recall@K, MRR or nDCG with annotated queries instead of tuning by anecdote.

Q5. When would you select TF-IDF over BERT for a production classifier?
Answer: When latency, cost, explainability and maintainability dominate, labels are limited and lexical patterns are strong. Benchmark macro-F1, rare-class recall, confidence calibration, robustness to new terminology and per-request cost. A sparse linear pipeline is a legitimate production choice if it meets the acceptance criteria.

---

3. Word Embeddings: Word2Vec and FastText

3.1 Core Algorithm and Workflow Intuition

How it works. Instead of a very wide sparse vector with one slot per vocabulary term, each word receives a short dense vector. Training exploits **distributional context**: words in similar surroundings acquire similar vectors. Word2Vec has two common objectives: **CBOW** predicts a center word from its neighbors; **Skip-gram** predicts neighboring words from a center word. Negative sampling makes training efficient by learning from true word-context pairs and sampled noise pairs. Unlike transformers, the result is normally a **static** embedding: bank has one vector even in “river bank” and “bank loan”.

FastText’s improvement: split words into character n-grams and compose word vectors from subword information. This helps misspellings, inflected words and many OOV terms. It does **not** solve contextual polysemy, and substring matches may be misleading. A sentence/document can be approximated by averaging word vectors, but that discards order and gives poor handling of negation.

Pros. Compact dense vectors, useful semantic neighborhoods, efficient small-model features and less sparse than TF-IDF. FastText extends coverage to many unseen spellings.

Cons / limits. Static vectors conflate meanings, reflect corpus bias, need meaningful training data, struggle with composition ("not good") and drift when domain vocabulary evolves. Averaging hides sentence structure.

When to choose:

1. **On-device or low-cost text categorization** where a full transformer is unnecessary.
2. **Specialized vocabularies / morphologically rich text**, especially with FastText subword n-grams.
3. **Semantic query expansion** or clustering for relatively stable domain language.

3.2 Architectural Blueprint

```
Corpus: "refund delayed order" / "return damaged item"
                          |
                 sliding context window
                          |
               Word2Vec CBOW / Skip-gram
                          |
                word -> fixed dense vector
                          |
           "refund"  [0.2, -0.1, 0.8, ...]

FastText: "refunding" -> <re, ref, efu, fun, ... , ing>
                          |
                     subword vector pooling
                          |
                usable representation for
                  many unseen surface words
```

3.3 Production-Ready Simple Code

Dependencies: pip install gensim numpy (a compatible NumPy/SciPy build is required).

```python
from gensim.models import Word2Vec, FastText
import numpy as np

sentences = [
    "payment refund pending", "refund requested yesterday",
    "payment failed order", "refund the payment",
    "shipment tracking delayed", "delivery delayed today",
    "order tracking available", "shipment arrived yesterday",
    "customer requested refund", "order delivery completed",
]
corpus = [s.split() for s in sentences]

# sg=1 -> Skip-gram; sg=0 -> CBOW. Single worker aids reproducibility.
w2v = Word2Vec(corpus, vector_size=50, window=3, min_count=1,
               sg=1, epochs=100, seed=42, workers=1)
ft = FastText(corpus, vector_size=50, window=3, min_count=1,
              sg=1, epochs=100, seed=42, workers=1)

# Word2Vec has no vector for an unseen surface word by default.
new_word = "refundings"
print("In Word2Vec vocabulary?", new_word in w2v.wv.key_to_index)
print("FastText OOV vector length:", len(ft.wv[new_word]))

# Document representation for a lightweight downstream model.
def mean_embedding(words, vectors):
    known = [vectors[w] for w in words if w in vectors.key_to_index]
    return np.mean(known, axis=0) if known else np.zeros(vectors.vector_size)

print("Document vector shape:", mean_embedding(corpus[0], w2v.wv).shape)
print("Nearest to 'refund':", w2v.wv.most_similar("refund", topn=2))
```

Production hardening: train on a much larger representative corpus, use held-out similarity and downstream task evaluation, freeze/version vectors, monitor corpus/domain drift, and evaluate fairness. Toy nearest-neighbor outputs are not meaningful.

3.4 Top 5 NLP Interview Questions

Q1. Explain CBOW versus Skip-gram and your choice for rare technical terms.
Answer: CBOW predicts the center from surrounding words; it tends to be efficient and smooth over frequent patterns. Skip-gram predicts contexts from a center term and can provide more useful rare-word representations with sufficient examples. For very sparse terms, FastText subwords or contextual pretrained models may be better; choose using downstream evaluation.

Q2. Why does FastText support OOV words while Word2Vec does not?
Answer: Word2Vec stores vectors for learned vocabulary entries. FastText also learns character n-gram vectors, so an unseen word can be composed from familiar fragments. OOV coverage is not semantic correctness: random strings may receive vectors but should not be treated as reliably understood entities.

Q3. Why are static embeddings weak for polysemy?
Answer: A word such as “charge” is represented similarly in “battery charge” and “fraudulent charge” because its vector is fixed, though contexts differ. Disambiguation must be handled by the downstream sequence model or by contextual token embeddings from transformers. Also distinguish word similarity from sentence-level entailment.

Q4. How would you use word vectors for document classification?
Answer: Pool vectors with a mean or weighted mean and fit a logistic regression or tree-based model. Manage missing-token coverage and prevent learned vectors/statistics from seeing test text in evaluation. For negation, compositionality and long documents, compare with n-gram baselines and transformer embeddings.

Q5. Two embedding models return similar cosine scores but different retrieval quality. Why?
Answer: Cosine value alone is not an absolute relevance probability, and different spaces have different geometric distributions. Check training objectives, pooling, vector normalization, semantic domain, chunking and negative sampling. Compare on a labeled retrieval set using Recall@K and nDCG, not the numeric magnitude of scores across unrelated models.

---

4. Sequence Models: RNNs, LSTMs and GRUs

4.1 Core Algorithm and Workflow Intuition

How it works. A Recurrent Neural Network reads tokens **in order**. Each step combines the current word embedding with an evolving **hidden state** summarizing previous tokens. The final state can classify a sentence; per-step states can tag entities or predict next tokens. Parameters are reused at every position. Training uses **backpropagation through time**: the gradient passes backward through these repeated steps.

Why plain RNNs struggle: when information must travel across many time steps, gradients may shrink (vanish) or grow (explode), making long-distance dependencies hard to learn. LSTM adds a **cell state** and learned **input, forget and output gates** that decide what to write, remember and expose. GRU uses update/reset gates with a simpler state. These improve long-range training, but do not make memory unlimited. **Bidirectional** variants scan left-to-right and right-to-left and are helpful when the full input is available, but invalid for strict streaming causal prediction.

Implementation decisions:

- Convert text into IDs with an explicit PAD and UNK, embed tokens, then pass variable-length sequences through the RNN/LSTM.
- **Pack or mask padded positions.** Otherwise, hidden states can be corrupted by padding; enforce consistent left/right padding choices.
- Train with cross-entropy; use gradient clipping to control exploding gradients. Regularize with dropout, early stopping and held-out evaluation.
- Inspect sequence length distribution and truncation. Recurrent computation is inherently less parallelizable across time than transformer self-attention.

Pros. Preserves sequence order, supports streaming inference, smaller architectures can be computationally appropriate, and naturally handles token-by-token data.

Cons / limits. Slower sequential training, fragile long-range reasoning, padding bugs, gradient instability and difficulty exploiting very long contexts compared with well-engineered modern attention models.

When to choose:

1. **Streaming signals / on-device intent detection** with compact models and limited latency budget.
2. **Sequence labeling** on moderate-length inputs when a recurrent baseline meets performance and cost goals.
3. **Time-ordered multimodal text/event streams** where the state can update incrementally.

4.2 Architectural Blueprint

```
token_1 -> Embedding -> [LSTM cell] -> h1
                                |
```

```
token_2 -> Embedding -> [LSTM cell] -> h2
                                |
token_3 -> Embedding -> [LSTM cell] -> h3 -> Classifier

Inside each LSTM cell:
Previous cell memory ---> [FORGET gate] ---+
New candidate ----------> [INPUT gate] ----+--> New cell memory
New cell memory --------> [OUTPUT gate] -------> Hidden state

Bidirectional encoder:  forward  -------->
                        backward <--------
                        concatenate states
```

Whiteboard tip: Contrast a plain RNN's single hidden-state update with the LSTM's gated memory highway. Distinguish a **hidden state** from a persistent user-memory database.

4.3 Production-Ready Simple Code

Dependencies: pip install torch scikit-learn (short text classification; CPU-friendly).

```python
import torch
from torch import nn
from torch.nn.utils.rnn import pack_padded_sequence
from sklearn.model_selection import train_test_split
from sklearn.metrics import accuracy_score

torch.manual_seed(42)
examples = [
    ("refund the order", 1), ("payment failed", 1),
    ("charged twice", 1), ("cancel purchase", 1),
    ("need a refund", 1), ("invoice incorrect", 1),
    ("login not working", 0), ("reset password", 0),
    ("application crashed", 0), ("server unavailable", 0),
    ("cannot sign in", 0), ("authentication failed", 0),
]
# PAD=0 and UNK=1; fit vocabulary using training text only.
texts, labels = zip(*examples)
tr, te = train_test_split(list(range(len(texts))), test_size=0.33,
                          random_state=42, stratify=labels)
vocab = {"<PAD>": 0, "<UNK>": 1}
for i in tr:
    for w in texts[i].split():
        vocab.setdefault(w, len(vocab))

MAX_LEN = 6
def encode(indices):
    ids, lengths = [], []
    for i in indices:
        row = [vocab.get(w, 1) for w in texts[i].split()][:MAX_LEN]
        lengths.append(len(row))
        ids.append(row + [0] * (MAX_LEN - len(row)))
    return torch.tensor(ids), torch.tensor(lengths)

class LSTMClassifier(nn.Module):
    def __init__(self, vocab_size):
        super().__init__()
        self.embedding = nn.Embedding(vocab_size, 24, padding_idx=0)
        self.lstm = nn.LSTM(24, 32, batch_first=True)
        self.head = nn.Linear(32, 2)

    def forward(self, ids, lengths):
        embedded = self.embedding(ids)
        packed = pack_padded_sequence(embedded, lengths.cpu(),
                                      batch_first=True, enforce_sorted=False)
        _, (last_hidden, _) = self.lstm(packed)
        return self.head(last_hidden[-1])

model = LSTMClassifier(len(vocab))
optimizer = torch.optim.Adam(model.parameters(), lr=0.01)
Xtr, Ltr = encode(tr)
ytr = torch.tensor([labels[i] for i in tr])
for epoch in range(25):
    model.train()
    optimizer.zero_grad()
    loss = nn.CrossEntropyLoss()(model(Xtr, Ltr), ytr)
    loss.backward()
    nn.utils.clip_grad_norm_(model.parameters(), max_norm=1.0)
    optimizer.step()

Xte, Lte = encode(te)
model.eval()
with torch.no_grad():
    predictions = model(Xte, Lte).argmax(dim=1).tolist()
print("Held-out accuracy:", accuracy_score([labels[i] for i in te], predictions))
```

Production hardening: fit a robust tokenizer on train data, batch with true-length packing, compare GRU/BiLSTM/transformer baselines, check long-text truncation, pin vocabulary and checkpoint, and monitor latency and class-specific error rates. The tiny demo is only a working architecture example.

4.4 Top 5 NLP Interview Questions

Q1. Why do vanilla RNNs suffer vanishing gradients, and how does an LSTM help?
Answer: Backpropagating through many sequential state updates repeatedly multiplies gradient factors; they can become very small or large. An LSTM has gated cell memory that provides a more stable path for important information, though long-range capture remains imperfect. Use gradient clipping, careful initialization, sequence-length checks and model comparison rather than assuming gating eliminates all instability.

Q2. LSTM training loss decreases but validation loss rises. What is your response?
Answer: Treat this as overfitting or distribution mismatch. Verify the split has no duplicate documents/users and compare error slices; then add regularization, early stopping, dropout between layers, or reduce hidden size. Validate that vocabulary fitting and preprocessing did not leak training-specific artifacts into evaluation.

Q3. What is the difference between LSTM and GRU?
Answer: LSTM has separate cell/hidden states and three functional gates; GRU combines memory mechanisms using a smaller set of gates and one main recurrent state. GRUs can be faster and more parameter-efficient, but superiority is empirical and workload-dependent. Test on the same sequence lengths, latency hardware, and validation protocol.

Q4. Why can padding silently ruin an RNN text classifier?
Answer: With ordinary right padding, the state at the last tensor position may represent multiple PAD tokens rather than the last real token. Packed sequences, appropriate masks, or gathering the state at the true length avoids that error. Audit empty inputs, unusually long inputs, PAD/UNK IDs and batch ordering.

Q5. When is bidirectional LSTM invalid?
Answer: It uses future tokens when encoding the current position, so it is inappropriate for strict real-time next-token prediction or causal streaming without future context. It is valid for offline document classification or NER where the whole sentence is available. Explain explicitly whether the task permits look-ahead.

---

5. Transformers: Attention, BERT and GPT

5.1 Core Algorithm and Workflow Intuition

How it works — modern NLP's key shift. A transformer tokenizes text into subwords, maps token IDs to embeddings, injects position information and passes vectors through repeated **self-attention** and feed-forward blocks. In self-attention, every token forms a *query* describing what information it needs, a *key* indicating what it offers, and a *value* carrying information. Compatibility between queries and keys yields attention weights; weighted values produce context-enriched token states. **Multi-head attention** learns multiple relationship patterns in parallel. Residual paths and layer normalization stabilize deep stacks.

Encoder versus decoder:

- **BERT-style encoder** attends bidirectionally to input tokens (subject to padding masks), generating contextual features for classification, token tagging, semantic matching and extractive QA. Original BERT pretraining is masked-language modeling (with sentence-level objectives in the original formulation).
- **GPT-style decoder** uses a **causal mask**: position *t* cannot attend to future positions. It repeatedly predicts the next token, appends it to the context and continues, which is appropriate for open-ended text generation and instruction following after suitable tuning.
- **Encoder-decoder** models (e.g., T5/BART) encode the source, then the decoder attends to source representations while autoregressively generating a target; useful for structured translation and summarization.

Core production choices: tokenizers and checkpoints must match; set truncation/max length consciously; use attention masks for padding and causal masks for generation; batch requests by approximate token length; monitor input/output token counts; prefer cached keys/values (**KV cache**) for autoregressive generation. Full self-attention often has quadratic cost in sequence length, though optimized/sparse/sliding-window variants change practical scaling.

Pros. Strong contextual disambiguation, highly parallel training over token positions, powerful transfer from pretrained checkpoints, and flexible classification/embedding/generation capabilities.

Cons / limits. Significant compute/memory, bounded effective context, susceptibility to prompt injection and hallucination, dataset bias, loss of middle-context reliability in some settings, and expensive long-output decoding. Attention weights alone are **not** a reliable causal explanation of a prediction.

When to choose:

1. **BERT/encoder:** domain intent classification, NER, reranking or high-quality sentence representation (with a model actually trained for sentence embeddings).
2. **GPT/decoder:** conversational assistants, drafting, structured extraction with constrained schema checks, tool-planning and text generation.
3. **Encoder-decoder:** translation, summarization or controlled rewriting with strongly aligned source and target text.

5.2 Architectural Blueprint

```
Input text -> Model tokenizer -> token IDs + positions
                           |
                      Embedding layer
                           |
                  +--------------------+
                  | Multi-head attention|
                  | + residual / norm  |
                  | Feed-forward / norm|
                  +--------------------+
                           | repeated N blocks
            +---------------+---------------+
            |                               |
        BERT ENCODER                    GPT DECODER
     All valid tokens may             Causal attention:
      attend to each other            no look-ahead
            |                               |
    [CLS] / pooled states             next-token logits
            |                               |
     class / tag / rank              sample / select token
                                            |
                                    append and repeat

Attention matrix (allow = X):
              key 1  key 2  key 3
 BERT query 1    X      X      X
      query 2    X      X      X
      query 3    X      X      X

 GPT query 1     X      -      -
     query 2     X      X      -
     query 3     X      X      X
```

Whiteboard tip: Label padding mask versus causal mask; distinguish a *pretrained language model* from a *sentence embedding model*. BERT is not inherently a good search embedding system without the right training/pooling.

5.3 Production-Ready Simple Code

Dependencies: pip install transformers torch scikit-learn. First execution downloads model weights from Hugging Face (requires network and enough disk/RAM).

```python
from transformers import pipeline
from sklearn.metrics import classification_report

# Encoder-family checkpoint fine-tuned for sentiment classification.
classifier = pipeline(
    "text-classification",
    model="distilbert/distilbert-base-uncased-finetuned-sst-2-english",
    device=-1,  # CPU; change only after checking available hardware
)
texts = ["The service was excellent.", "The delivery was terrible."]
truth = ["POSITIVE", "NEGATIVE"]
results = classifier(texts, truncation=True, max_length=256)
predicted = [item["label"] for item in results]
print(classification_report(truth, predicted, zero_division=0))

# Decoder-only generation; deterministic decoding for reproducibility.
generator = pipeline("text-generation", model="distilbert/distilgpt2", device=-1)
reply = generator(
    "Good customer support means",
    max_new_tokens=30,
    do_sample=False,
    pad_token_id=generator.tokenizer.eos_token_id,
)
print(reply[0]["generated_text"])
```

Production hardening: test tokenizer compatibility, calibration for classification, max context under real traffic, safety checks, citation/grounding evaluation, output validation, deterministic version pinning and cost/latency. For semantic search, use a retrieval-optimized embedding model rather than arbitrary raw BERT outputs.

5.4 Top 5 NLP Interview Questions

Q1. Explain self-attention without writing the equation. Why multiple heads?
Answer: Each token asks which other tokens are useful, scores their relevance, and blends information from them into an updated contextual representation. Separate heads learn complementary patterns such as local syntax, entity references or long-distance cues; they are not guaranteed to map neatly to human-interpretable relationships. Position information is essential because attention alone has no built-in token ordering.

Q2. Why is BERT bidirectional but GPT autoregressive?
Answer: BERT-style encoding can use left and right input context because it is learning a representation of already-available text. GPT prevents access to future targets via a causal mask so next-token generation is a valid prediction problem. This difference affects objectives and ideal downstream tasks, not a blanket assertion that one model is always better.

Q3. How would you debug an LLM that ignores information near the middle of a long prompt?
Answer: Check real token counts, truncation, prompt template, retrieval ordering and whether evidence is buried in repeated boilerplate. Run location-sensitivity tests by moving identical facts to the beginning, middle and end; rerank and compress relevant context, or retrieve smaller coherent passages. Bigger context alone does not ensure effective use of every token.

Q4. What is the KV cache and what trade-off does it introduce?
Answer: During autoregressive decoding, the model reuses stored attention keys and values for previous tokens rather than recomputing those projections every step. This accelerates incremental generation but uses memory that grows with active sequence length and concurrent requests. Production serving balances cache memory, batch size, context length, throughput and latency.

Q5. A classifier's accuracy is high but minority-class recall is poor. What do you change?
Answer: First inspect the confusion matrix, class-wise precision/recall, dataset provenance and label noise. Optimize macro-F1 or minority recall at an acceptable precision; consider stratified sampling, class-weighted loss, threshold calibration and targeted new examples. Evaluate on representative deployment slices rather than reporting accuracy alone.

---

6. LLM Fine-Tuning: SFT, LoRA, QLoRA and Evaluation

6.1 Core Algorithm and Workflow Intuition

What is being adapted? A base LLM learns broad language patterns in pretraining. **Supervised fine-tuning (SFT)** then teaches desired prompt-to-answer behavior from demonstrations. For each training example, text is formatted using the correct model chat template; the tokenizer converts it into IDs; next-token prediction loss updates selected parameters on the desired answer tokens. A separate validation set guards against memorization and regressions. For chat models, correct role delimiters, end-of-turn tokens and completion-only loss masking are critical.

Adaptation strategies:

Method	What changes	Relative cost	Best reason to choose
Prompting / few-shot	No weights; instruction and examples in context	Very low upfront	Behavior is achievable with prompting
RAG	Retrieves external evidence during inference	Index + retrieval overhead	Facts change or require citation/access control
Full fine-tuning	Most/all pretrained weights	Highest memory/compute	Large, justified domain shift and capacity
LoRA (PEFT)	Small trainable low-rank matrices in selected layers	Lower trainable parameter count	Cheap, modular behavior/domain adaptation
QLoRA	Quantized base model + LoRA adapters	Lower training-memory footprint	Larger-model adaptation on limited GPU memory
Preference tuning (e.g. DPO)	Model updated from preferred vs rejected responses	Additional preference data	Ranking/refining response behavior

Critical boundaries: Fine-tuning is mainly for **behavior, format, style, domain procedure, or repeated patterns**; it is usually a poor primary mechanism for frequently changing facts that need sourcing. RAG and fine-tuning can be

complementary. Quantization is not training by itself. LoRA adapters reduce trainable parameters, but the frozen base still consumes inference/activation memory.

A defensible workflow:

1. Define measurable requirements and establish a prompt/RAG baseline **before** training.
2. Curate, license and de-identify examples; deduplicate; split by customer/document/time and keep a truly untouched test set.
3. Verify prompt and response format; use the model's tokenizer and chat template; ensure padding and labels are masked correctly.
4. Select base model and LoRA target modules from its architecture; set sequence length, optimizer, learning rate, effective batch size and checkpoint policy.
5. Train with validation, monitor loss, gradient norm, token budget and catastrophic forgetting; early-stop or pick the best checkpoint.
6. Evaluate on golden domain tasks, general-capability regression tests, refusal/safety behavior, groundedness, latency, cost and real-user acceptance.
7. Version the base model, tokenizer, adapter, dataset fingerprint and evaluation report; deploy with canary and rollback.

Pros. Improves consistent response formatting and domain conventions without repeating large few-shot prompts; adapters are comparatively cheap to train, swap and store.

Cons / limits. Quality depends strongly on demonstration quality; risk of overfitting, forgetting, contamination, unsafe imitation, hardware cost, model-license restrictions and mismatches between trained chat template and serving prompts.

When to choose:

1. **Structured extraction at scale:** consistently emit strict schema fields from domain-specific notes when prompting alone underperforms.
2. **Enterprise assistant response discipline:** stable answer format, controlled terminology and workflow behavior while RAG handles changing policy facts.
3. **Specialized instruction following:** repeated domain tasks with thousands of high-quality, consented examples and robust evaluation.

6.2 Architectural Blueprint

```
      Prompt / RAG baseline + success metrics
                        |
Licensed, redacted prompt-response examples
                        |
       Train / validation / TEST split
                        |
      Format with correct chat template
                        |
         Tokenizer + target loss mask
                        |
Frozen pretrained weights (possibly 4-bit)
                        |
Small TRAINABLE LoRA matrices in projections
                        |
      Loss on desired response tokens
                        |
      Validate + regression/safety suite
                        |
   Version adapter, base, tokenizer, data
                        |
          Canary -> monitor -> rollback
```

Whiteboard tip: Draw the frozen base as a large gray block and LoRA adapters as small attached rectangles. Draw **retrieved documents** entering at inference time, not being magically stored in weights.

6.3 Production-Ready Simple Code

Dependencies: pip install torch transformers peft. This is a **small, runnable LoRA learning exercise**, not a production-quality trained LLM. Download of the tiny model requires internet.

```
import torch
from transformers import AutoTokenizer, AutoModelForCausalLM
from peft import LoraConfig, get_peft_model

torch.manual_seed(42)
model_id = "sshleifer/tiny-gpt2"  # Tiny demo checkpoint, NOT a capable LLM
```

```
base = AutoModelForCausalLM.from_pretrained(model_id)
tok = AutoTokenizer.from_pretrained(model_id)
tok.pad_token = tok.eos_token
base.config.use_cache = False  # Training: do not need decode cache

# GPT-2 uses 'c_attn'; other architectures have different target names.
config = LoraConfig(
    task_type="CAUSAL_LM", r=8, lora_alpha=16, lora_dropout=0.05,
    target_modules=["c_attn"], fan_in_fan_out=True,
)
model = get_peft_model(base, config)
model.print_trainable_parameters()

examples = [
    ("Question: What is RAG?\nAnswer:",
     " Retrieval-augmented generation uses external documents."),
    ("Question: What is LoRA?\nAnswer:",
     " LoRA trains small low-rank adapter matrices."),
]
texts = [p + a + tok.eos_token for p, a in examples]
inputs = tok(texts, padding=True, truncation=True,
             max_length=96, return_tensors="pt")
labels = inputs["input_ids"].clone()

# Learn from answer tokens only, not the instruction or PAD tokens.
for i, (prompt, _) in enumerate(examples):
    prompt_len = len(tok(prompt, add_special_tokens=False)["input_ids"])
    labels[i, :prompt_len] = -100
labels[inputs["attention_mask"] == 0] = -100

optimizer = torch.optim.AdamW(
    [p for p in model.parameters() if p.requires_grad], lr=2e-4
)
model.train()
for step in range(3):  # Illustrative steps, not meaningful convergence
    optimizer.zero_grad()
    loss = model(**inputs, labels=labels).loss
    loss.backward()
    torch.nn.utils.clip_grad_norm_(model.parameters(), 1.0)
    optimizer.step()
    print("Step:", step, "loss:", round(loss.item(), 3))

model.save_pretrained("./demo_lora_adapter")
tok.save_pretrained("./demo_lora_adapter")
```

Production hardening: use a genuinely capable licensed model, audited datasets, supported chat templates, dedicated validation set, correct model-specific target modules, mixed precision/quantization on supported hardware, checkpoint/gradient accumulation strategy, answer-only masks, safety regression tests and adapter/base compatibility checks. Some checkpoints add special tokens around roles; always test label alignment explicitly.

6.4 Top 5 NLP Interview Questions

Q1. When should a team use RAG instead of fine-tuning?
Answer: Prefer RAG for changing factual knowledge, source citations, permissioned documents and auditable freshness. Prefer SFT/LoRA for stable response behavior, formatting and domain task skills that prompting cannot reliably elicit. Combine them when the assistant needs both up-to-date evidence and consistent behavior; prove each component improves the right metric.

Q2. Explain LoRA versus QLoRA and the memory savings honestly.
Answer: LoRA freezes the base weights and trains small low-rank update matrices, reducing optimizer and gradient memory. QLoRA additionally keeps the base model quantized (commonly 4-bit) during adapter training, lowering base-weight storage. Activations, KV cache, optimizer state for adapters and sequence length still consume memory; quality and speed require empirical validation.

Q3. Training loss improves but downstream answers hallucinate more. What happened?
Answer: Lower next-token loss on training demonstrations does not guarantee truthful or grounded outputs. The dataset may reward confident unsourced answers, contain contamination, omit abstention examples or mismatch the inference retrieval workflow. Evaluate factuality/faithfulness against evidence, refusal behavior and general regression sets, then improve labels, retrieval and training objectives rather than blindly extending training.

Q4. What causes silent fine-tuning failures with chat templates and label masks?
Answer: Wrong role delimiters, EOS/end-of-turn handling, padding tokens, tokenization concatenation and shifts can teach the model to mimic prompt text or ignore completion targets. Inspect a decoded minibatch and token-level loss mask; assert that answer tokens, and only intended tokens, contribute to loss. Run an overfit-a-tiny-batch sanity test before long jobs.

Q5. How would you decide whether a fine-tuned adapter is production-ready?
Answer: Set quantitative acceptance gates: schema validity, task success, refusal safety, answer groundedness, rare-case recall, domain benchmarks, general regression performance, p95 latency and cost. Compare against the best prompt/RAG baseline on an untouched holdout, conduct human review on failure slices and canary the release. Keep exact versioned artifacts and a rollback path.

Cross-Model Decision Table — Explain Your Choice in 60 Seconds

Need	First model to benchmark	Upgrade trigger	Evaluation to trust
Fast ticket labels	TF-IDF + logistic regression	Too many paraphrase/syntax failures	Macro-F1 and per-class recall
Rare spellings and morphology	FastText-based features	Context changes word meaning	OOV slices + downstream F1
Small streaming sequences	GRU/LSTM	Long-range dependencies / slower training	F1 vs p95 latency and RAM
Document classification / NER	Encoder model	Need open-ended generation	Macro-F1 / span-level F1
Text generation	GPT-style decoder	Facts need citation or update	Task success + factuality + cost
Changeable private knowledge	Hybrid search + RAG	Stable response-format errors	Recall@K + grounded answer rate
Repeated narrow assistant behavior	Prompting, then LoRA if justified	Behavior still inconsistent	Golden set + safety regression

Senior interview answer template: “I would start with [simple baseline] because [latency/data/interpretability], measure [primary metric and hard constraint], inspect [failure slices], then upgrade to [alternative] only if the improvement justifies [cost, maintenance and risk].”

Ultimate NLP Interview Cheat Sheet — 30 Most-Asked Questions

The following is deliberately standalone: each answer is formulated as a concise, senior-level interview response with the decision rule or practical caveat that interviewers look for.

A. Data, Training and NLP Lifecycle

01. Describe the end-to-end NLP production lifecycle.
Answer: Define the user task, quality metric, latency/cost targets and data/privacy constraints first. Audit and label representative text; split by user/document/time where appropriate; establish an interpretable baseline; train and evaluate on unseen slices; package preprocessing, tokenizer and model together. In production, monitor drift, quality proxies, safety, cost, latency and data retention; retrain or roll back with traceable versioning.

02. What is data leakage in NLP, and why is duplicate text a common trap?
Answer: Leakage occurs when training learns information unavailable at deployment or from evaluation records. Repeated customer tickets, near-identical templates and related chunks can appear on both sides of a random split, inflating scores. Use user/document/time-grouped splits, near-duplicate detection and a truly untouched test set; fit vocabulary, IDF and learned preprocessors only on train data.

03. How do you handle class imbalance in intent classification?
Answer: Inspect per-class support, confusion matrices and business cost. Prefer macro-F1, precision/recall trade-offs, PR-AUC or recall at constrained precision over accuracy alone; use stratified validation, class weights, targeted collection and threshold tuning on validation only. Do not blindly oversample before splitting, because near-duplicates can leak.

04. How do you diagnose data drift versus concept drift?
Answer: Data/covariate drift changes input distributions, such as new slang, languages or document formats; concept drift changes the mapping from text to the correct label. Track token and embedding distributions, OOV/truncation rates and human-labeled outcome quality. If labels or task definitions changed, updating only the tokenizer will not restore quality; retraining, relabeling or policy revision may be necessary.

05. What are effective ways to augment an NLP dataset safely?
Answer: Candidate techniques include paraphrases, back-translation, entity substitutions and synthetic examples, but all can alter meaning or expose sensitive information. Apply augmentations to the training split only, preserve label-defining spans (especially negation, numbers, units and names) and manually review a sample. Verify improvements on independent real-world data, not generated examples that share the same artifacts.

B. Evaluation and Statistical Interpretation

06. How do precision, recall, F1 and accuracy differ?
Answer: Precision is the share of predicted positives that are correct; recall is the share of real positives found; F1 balances the two. Accuracy is useful for balanced, symmetric-error classification but can conceal poor minority-class coverage. Match the primary metric to the cost of false positives versus false negatives, and report per-class results.

07. What are BLEU and its important limitations?
Answer: BLEU measures overlap of candidate and reference n-grams, with a brevity penalty, and was designed primarily for corpus-level machine translation evaluation. Valid paraphrases can score poorly, while fluent but semantically wrong text can score deceptively well. Use suitable tokenization and multiple references when possible; combine with meaning/factuality assessment and human judgments.

08. What is ROUGE, and when would you use ROUGE-L?
Answer: ROUGE is a family of reference-overlap metrics often used for summarization. ROUGE-N measures n-gram overlap; ROUGE-L uses longest common subsequence to reflect in-order overlap. These are useful regression indicators but cannot establish factual correctness, coverage of critical claims or whether the summary contradicts its source.

09. What does perplexity tell you about a language model?
Answer: Perplexity summarizes how surprised a language model is by held-out token sequences: lower generally means better next-token prediction on that test distribution. It is sensitive to tokenization and evaluation corpus, so raw numbers across different tokenizers/models may not be comparable. A low perplexity model can still produce unsafe or ungrounded responses, making task-level evaluation necessary.

10. How would you evaluate a generated answer when no single reference answer exists?
Answer: Build a rubric for task completion, grounded factual claims, missing critical information, format compliance and safety; use adjudicated human examples as an anchor. Automated semantic/LLM-judge scores can scale triage but require calibration for position bias, verbosity bias and judge-model failure. Report agreement rates, error slices and confidence intervals where sampling permits.

C. Architectures, Context and Representations

11. Compare sparse TF-IDF vectors, static embeddings and contextual embeddings.
Answer: TF-IDF captures weighted lexical overlap and is transparent but misses synonyms. Word2Vec/FastText maps each word to a dense vector and can generalize semantically, but usually assigns one embedding per word regardless of sentence context. Transformer representations depend on surrounding tokens, making them stronger for ambiguity and compositional tasks at higher compute cost.

12. What is self-attention, and why is positional information necessary?
Answer: Attention mixes each token's information with information from other tokens according to learned relevance scores. Without position signals, a standard attention block cannot directly distinguish order permutations of the input. Positional embeddings or rotational/relative schemes let the architecture represent sequence order; design choices affect length generalization.

13. What is the difference between an attention mask and a causal mask?
Answer: An attention/padding mask prevents processing padded or otherwise inaccessible tokens; a causal mask prevents each target position from seeing future tokens. An encoder commonly uses bidirectional attention over valid input, while an autoregressive decoder uses causal masking. Incorrect masks cause subtle leakage, poor output quality and misaligned training/inference behavior.

14. What does the context window mean, and is longer always better?
Answer: The context window bounds how many tokens a specific checkpoint can process in one inference call, subject to positional and serving constraints. Longer prompts consume memory/time and may bury relevant facts; effective context use is not guaranteed by the advertised limit. Retrieve and order salient evidence, compress safely, monitor truncation and test location-sensitive questions.

15. Why can't arbitrary BERT token vectors be used as high-quality sentence embeddings?
Answer: Raw token outputs or an untrained mean pool may not form a space where cosine distance aligns with semantic relevance. Sentence embedding models are explicitly trained with positive/negative pairs and pooling choices for retrieval or similarity. Validate the chosen encoder on labeled query-document matches and normalize vectors consistently with the index.

D. Search, Vector Databases and RAG

16. Compare lexical search, dense vector search and hybrid retrieval.
Answer: Lexical systems (e.g., BM25) excel at exact terminology, numbers, codes and unusual proper nouns; dense search excels at paraphrases and semantic similarity. Hybrid retrieval combines candidate sets or scores so one method's blind spots are covered by the other. Tune fusion on real judged queries and track Recall@K before adding expensive reranking.

17. How do you choose chunk size and overlap for document RAG?
Answer: Start from document structure and answer granularity, not a universal token count. Preserve table boundaries, headings and citation offsets; overlap only enough to reduce boundary loss, because excess overlap creates duplicates and increases indexing/serving cost. Run retrieval and answer-quality experiments over chunk size, overlap and hierarchical context expansion.

18. What are Recall@K, MRR and nDCG in retrieval?
Answer: Recall@K asks what fraction of relevant evidence appears among the top K retrieved items. MRR rewards the rank of the first relevant result; nDCG evaluates ranking quality using graded relevance and position discounts. Use them with a query-level relevance set, and separate retrieval performance from whether the answer generator actually used the evidence.

19. Why use approximate nearest neighbor indexing such as HNSW?
Answer: Exact search scans many vectors and becomes expensive at large scale. HNSW builds a navigable proximity graph for fast approximate retrieval, trading memory/build time and tuning parameters for recall and latency. Benchmark p95 latency, Recall@K, index update patterns, metadata-filter selectivity and operational constraints under production-like load.

20. What is reranking and when is it worth the latency?
Answer: First-stage search retrieves a broad set of candidate chunks cheaply; a cross-encoder or LLM-based reranker then scores query-document pairs jointly for more precise ordering. It helps when the correct evidence is retrieved but buried among near matches. Rerank a bounded candidate set and measure answer improvement against extra p95 latency and cost.

E. Generative Models, Adaptation and Serving

21. RAG versus fine-tuning: what problem does each solve?
Answer: RAG supplies external facts, freshness, provenance and per-user permissions at inference time. Fine-tuning changes the model's learned behavior or domain task ability and does not offer reliable real-time knowledge updates. Many enterprise systems benefit from both, but each should pass a distinct ablation test against a strong prompt-only baseline.

22. What are the practical differences between full fine-tuning, LoRA and QLoRA?
Answer: Full tuning updates most weights and needs the largest optimizer/gradient budget; LoRA trains a comparatively small set of adapter matrices while freezing the base. QLoRA also stores the frozen base in lower precision during adapter training to reduce memory. Select based on task difficulty, supported hardware, adapter compatibility, inference requirements and validated downstream quality.

23. What is instruction fine-tuning versus preference tuning?
Answer: Instruction/SFT learns from examples of desired responses, teaching formats and task behaviors. Preference methods (such as DPO) use comparisons between preferred and rejected outputs to shape relative response behavior. Neither guarantees factuality or safety without high-quality labels, explicit policies and downstream tests.

24. How do temperature, top-k and top-p affect generation?
Answer: Temperature changes how peaked token-selection probabilities are; a lower temperature often yields more repeatable outputs, not proof of correctness. Top-k restricts choice to the k highest-scoring tokens and top-p to the smallest set exceeding a cumulative probability threshold. For structured tasks, test deterministic settings plus schema validation; sampling parameters alone cannot enforce constraints.

25. How would you improve LLM latency while preserving answer quality?
Answer: Decompose total time into retrieval, reranking, prompt construction, prefill, decoding and tool calls; inspect p50/p95/p99 rather than one average. Reduce unnecessary context/output tokens, cache stable computations, batch/prefix-cache where valid, use quantization or smaller models when quality permits, and stream the first tokens. Recheck quality and tail latency at production concurrency.

F. Reliability, Security and Senior-Level System Design

26. What is a hallucination, and how do you reduce it in enterprise NLP?
Answer: A hallucination is unsupported or false generated content presented as if established. Limit responses to authorized evidence with explicit citation requirements, use retrieval quality checks, teach abstention when evidence is insufficient, and validate factual claims where feasible. Evaluate groundedness and correctness separately: a fluent cited answer can still misinterpret the source.

27. What is prompt injection, including indirect prompt injection?
Answer: Prompt injection is an attempt to override the assistant's intended instruction hierarchy; indirect injection arrives through lower-trust material such as retrieved webpages, PDFs, tool outputs or email. Treat retrieved content strictly as data, enforce tool authorization outside the model, constrain schemas, and require user approval for impactful actions. Tests should include adversarial documents that request data exfiltration or unauthorized tool use.

28. How do you protect sensitive data in NLP and vector retrieval?
Answer: Apply least-privilege access before retrieval, enforce tenant/ACL filters in indexes and databases, minimize/redact PII, encrypt data and limit retention. Retrieved documents, embeddings, logs and prompts can all contain or reveal sensitive information; do not assume vectors are anonymized. Audit access by user and document identity and test cross-tenant isolation.

29. How do you monitor an NLP service in production?
Answer: Measure service reliability (error rate, timeouts, p95 latency), workload (tokens, request mix, costs) and task quality (sampled human labels, drift, retrieval recall proxies, abstention and schema validity). Trace requests across preprocessing, search, reranking, model generation and tools with privacy-safe logging. Use slice-based alerts and repeatable eval datasets to distinguish a model regression from an upstream data issue.

30. Design an enterprise document-answering assistant. What would you build first?
Answer: Start with document authorization, OCR/structure-aware extraction, metadata/page citations and a clear benchmark of realistic user queries. Build lexical+dense hybrid retrieval, optional reranking, a constrained LLM response format and refusal when evidence is missing; evaluate retrieval recall separately from grounded answer correctness. Add access controls, prompt-injection defenses, observability, freshness/reindex triggers, latency/cost budgets, human escalation and a rollback plan before production rollout.

Last-Page Rapid Revision

Prompt you may hear	One-line answer to anchor your response
Tokenization versus embeddings?	Tokenization maps text to discrete IDs; embeddings map IDs or spans to learned vectors.
TF-IDF versus BM25?	Both lexical; TF-IDF weighs term rarity, BM25 adds tuned term saturation and document-length handling.
Word2Vec versus FastText?	Word2Vec learns whole-word static vectors; FastText also learns character subword vectors and helps OOV.
LSTM versus Transformer?	LSTM propagates recurrent state sequentially; Transformer uses attention to connect token positions.
BERT versus GPT?	Bidirectional encoder representation versus causal next-token generation.
Embedding model versus generative model?	Retrieval embeddings optimize similarity; generative models optimize token prediction and response behavior.
RAG versus SFT?	RAG injects fresh evidence; SFT adapts behavior and task conventions.
LoRA versus QLoRA?	Train low-rank adapters versus do so on a quantized frozen base.
BLEU / ROUGE / Perplexity?	Reference overlap for translation/summaries versus held-out language-model predictability.
Best retrieval metric?	Recall@K for evidence coverage, MRR/nDCG for ordering; evaluate generated answers separately.
Main production risks?	Leakage, wrong permissions, poor grounding, drift, latency/cost, unsafe tool use and weak evaluation.

Recommended practice: explain any algorithm in four moves: **input → transformation → learned representation → output**. Then volunteer **one limitation, one failure test and one fallback**. This is often more convincing than presenting equations without engineering consequences.

Official implementation references

These public documentation pages are useful when reproducing the examples. Library APIs evolve; pin compatible package versions and model revisions in an actual project.

1. spaCy tokenization and language pipelines: https://spacy.io/usage/linguistic-features
2. scikit-learn text feature extraction: https://scikit-learn.org/stable/modules/feature_extraction.html#text-feature-extraction
3. Gensim Word2Vec / FastText: https://radimrehurek.com/gensim/models/word2vec.html and https://radimrehurek.com/gensim/models/fasttext.html
4. PyTorch LSTM: https://docs.pytorch.org/docs/stable/generated/torch.nn.LSTM.html
5. Hugging Face Transformers pipelines: https://huggingface.co/docs/transformers/en/pipeline_tutorial
6. Hugging Face PEFT LoRA: https://huggingface.co/docs/peft/en/package_reference/lora
7. Hugging Face Trainer: https://huggingface.co/docs/transformers/en/main_classes/trainer

End of print-ready NLP Masterclass.